# MANUAL DE DESPLIEGUE

# Docker local y estrategia de nube

**Predictive Maintenance AI - Fase II / Módulo 2, Semana 8.**

Objetivo: instalar, ejecutar y verificar una aplicación académica de mantenimiento predictivo que estima el riesgo de al menos una falla en las próximas **72 horas** en doce máquinas sintéticas.

**Alcance:** máquina de desarrollo Windows 11 con Docker Desktop/WSL 2 o Linux con Docker Engine y Compose. La aplicación consta de dos contenedores y SQLite persistente. Ningún despliegue cloud comercial ha sido ejecutado.

## Resumen de arquitectura

| Componente | Servicio | Responsabilidad |
|---|---|---|
| Interfaz web | Streamlit :8501 | Riesgo de 12 máquinas, escenarios y alertas |
| API | FastAPI :8000 | Validación, inferencia y endpoints Swagger |
| Machine Learning | RandomForest | Riesgo futuro a 72 h con artefacto joblib |
| Persistencia | SQLite | Registro de inferencias y fecha de lectura simulada |
| Contenedores | Docker Compose | API y web con red interna y healthcheck |

**Restricción:** datos, equipos y eventos completamente sintéticos. No se afirma desempeño productivo ni seguridad industrial demostrada.

## 1. Requisitos y preparación del anfitrión

Windows 11, Docker Desktop con WSL 2 y Engine running, puertos locales 8000 y 8501 libres, acceso inicial para descargar las imágenes Python, Git o GitHub Desktop y recursos suficientes para los dos contenedores.

| Comprobación | Comando PowerShell | Criterio |
|---|---|---|
| Docker | docker --version | Cliente disponible |
| Compose | docker compose version | Plugin instalado |
| Motor real | docker run --rm hello-world | Hello from Docker! |
| Archivos | git clone [URL del repo] | Modelo, Dockerfiles, compose y código |

## 2. Ejecución completa local

Desde PowerShell, clonar el repositorio público https://github.com/isaaccg88/PredictiveMaintenance-AI-FaseII y ejecutar:

```
git clone https://github.com/isaaccg88/PredictiveMaintenance-AI-FaseII.git
cd PredictiveMaintenance-AI-FaseII
docker compose up --build -d --wait
docker compose ps
```

Abrir **http://localhost:8501** para el dashboard. Abrir **http://localhost:8000/docs** para el contrato API. El endpoint GET /health debe indicar status=ok, database=ok y model_loaded=true.

## 3. Cierre seguro sin perder registros

Usar **docker compose down** para cerrar. El volumen **prediction_state** conserva SQLite. No utilizar la opción '-v' si se quiere mantener el historial de predicciones.

## 4. Diseño de contenedores y conectividad

Dockerfile.api empaqueta FastAPI y el artefacto de modelo previamente entrenado. Dockerfile.web empaqueta Streamlit. Compose levanta la API primero y espera que el endpoint health responda antes de iniciar la interfaz. La UI consulta la API por el nombre de servicio **http://api:8000**, no por localhost dentro del contenedor.

Los puertos host usan la dirección 127.0.0.1, de modo que los servicios no se exponen intencionalmente a la red local. SQLite reside en **/state/predictions.sqlite** dentro de un volumen persistente. Ningún secreto se incluye en el contenedor.

## 5. Endpoints para comprobación

| Método y ruta | Uso | Resultado esperado |
|---|---|---|
| GET /health | Salud de API/modelo/BD | HTTP 200; model_loaded=true |
| GET /machines | Listado del piloto | 12 máquinas sintéticas |
| GET /telemetry/PM-049 | Sensores de una máquina | Lectura fechada, simulada |
| POST /predict | Predicción y registro | 72 h, probabilidad, ID de SQLite |
| GET /predictions | Auditoría de consultas | Historial persistente |
| GET /model/info | Trazabilidad | Umbral, características, condición sintética |

## 6. Casos de error previstos

Máquina ajena al piloto responde 404. Sensores fuera de los límites declarados o campos desconocidos responden 422. La API no acepta silenciosamente una máquina inexistente. El dashboard diferencia una nueva simulación hipotética de la última lectura generada.

## 7. Estrategia cloud: contenedores o PaaS

Se plantea una migración futura a un proveedor capaz de ejecutar imágenes OCI (por ejemplo Azure Container Apps, Cloud Run o AWS App Runner). Esta descripción no afirma que el proyecto esté desplegado ni que se hayan contratado servicios.

| Paso | Actividad técnica | Control / evidencia |
|---|---|---|
| 1 | Versionar una etiqueta Git inmutable | SHA del modelo y del código |
| 2 | Construir/publicar dos imágenes OCI | Registry con permisos mínimos |
| 3 | Crear servicios backend/frontend | HTTPS, rutas y healthchecks |
| 4 | Configurar variables y secretos | Secret manager, nunca en Git |
| 5 | Provisionar persistencia administrada | Migrar SQLite a PostgreSQL si hay réplicas |
| 6 | Pruebas de humo y carga | Predicción, 404/422, logs y latencia |
| 7 | Monitorear y controlar costos | Alertas, limites, presupuestos, rollback |

## 8. Requisitos adicionales antes de producción

Identidad y autenticación de técnicos, cifrado TLS, auditoría, autorizaciones por rol, control de versiones de modelos, métricas de calibración, drift, observabilidad, respaldo y recuperación, cumplimiento industrial y datos reales. La construcción de datos sintéticos sirve para aprendizaje y demostración técnica solamente.

## 9. Herramientas de documentación y declaración de IA

ChatGPT (OpenAI) apoyó el diseño de arquitectura, código, Docker y documentos; la ejecución y los resultados son verificables en GitHub Actions y, posteriormente, con capturas locales auténticas. El usuario conserva responsabilidad sobre la entrega y conclusiones.

## 10. Diagnóstico rápido y referencias

| Síntoma | Verificación / solución |
|---|---|
| API unhealthy | docker compose logs api; comprobar que artifacts/model.joblib está presente |
| Puertos ocupados | Revisar procesos de Windows o ajustar puertos host de Compose |
| Error frontend -> API | Confirmar API_URL=http://api:8000 dentro de Docker |
| Sin historial | Revisar volumen prediction_state y /state/predictions.sqlite |
| Modelo ausente | Revisar Actions: archivo serializado se publica tras CI aprobada |
| Pérdida de volumen | No ejecutar docker compose down -v salvo borrado intencional |

Comprobar el código de forma pública: PredictiveMaintenance-AI-FaseII (GitHub). Los logs y artefactos de CI están en la pestaña Actions. Docker, Compose y FastAPI cuentan con documentación oficial de sus respectivos proyectos.

**Conclusión:** la solución se empaqueta en dos imágenes Docker y admite ejecución conjunta con una sola orden, con API y persistencia. El manual prepara un despliegue cloud realista, pero se limita a una aplicación académica sin telemetría industrial.